BÀI 12: EXCEPTION HANDLING VÀ LOGGING TRONG NESTJS

Mục tiêu bài 12: Bài này sẽ giới thiệu đến các bạn về xử lý lỗi Exception trong Nestjs, giới thiệu về Logging và lưu dữ liệu logging vào tập tin trong Nestjs

12.1. EXCEPTION HANDLING

12.1.1. Giới thiệu Exception Handling

Exception Handling (xử lý ngoại lệ) là cơ chế giúp ứng dụng **NestJS**:

- Phản hồi lại các lỗi một cách có kiểm soát.
- Tránh để ứng dụng bị "crash".
- Gửi thông báo lỗi rõ ràng, có cấu trúc đến client (thường là JSON).
- Ghi log lỗi để phục vụ debug và theo dõi.

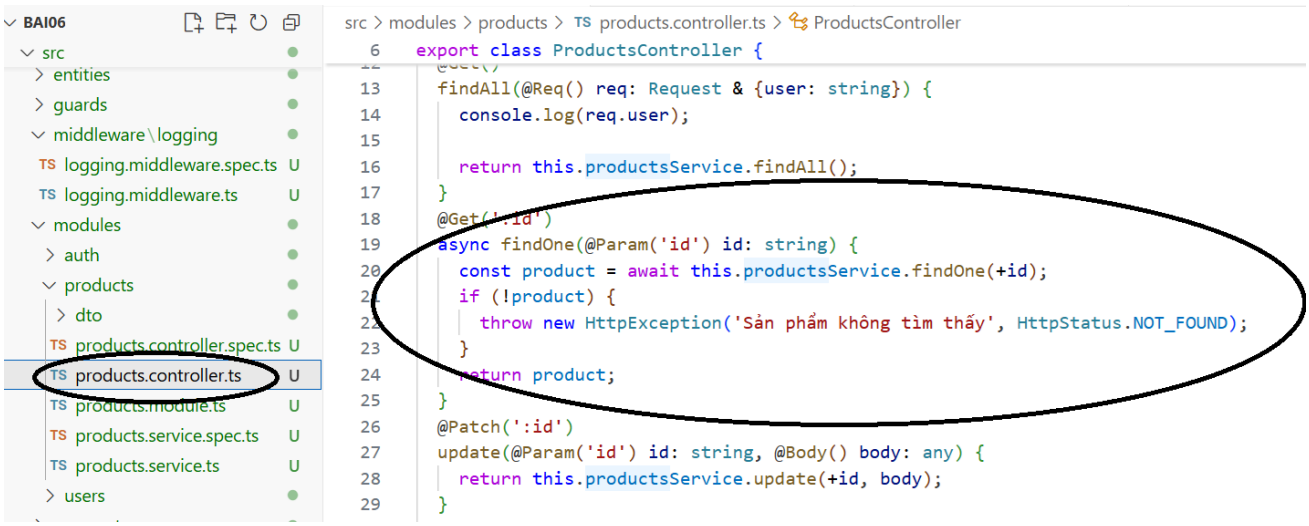
NestJS hỗ trợ sẵn nhiều lớp ngoại lệ thường dùng:

- throw new **BadRequestException**('Dữ liệu không hợp lệ');
- throw new **UnauthorizedException**('Chưa xác thực');
- throw new **NotFoundException**('Không tìm thấy tài nguyên');
- throw new **ForbiddenException**('Không có quyền truy cập');

12.1.2. Ví dụ về Exception Handling

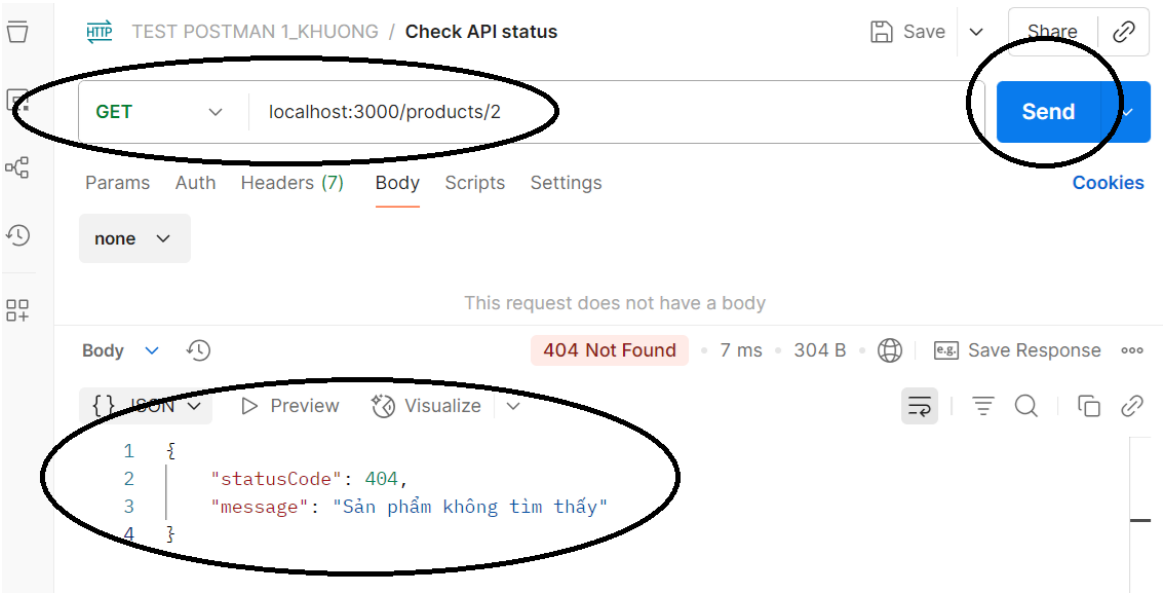
- Các bạn mở lại dự án ở bài trước ở VScode

- Các bạn mở tập tin **products.controller.ts** thì các bạn đã thấy chúng ta đã sử dụng **Exception** khi tìm kiếm sản phẩm thông qua **id**.



- Các bạn biên dịch chương trình: **npm run start:dev**

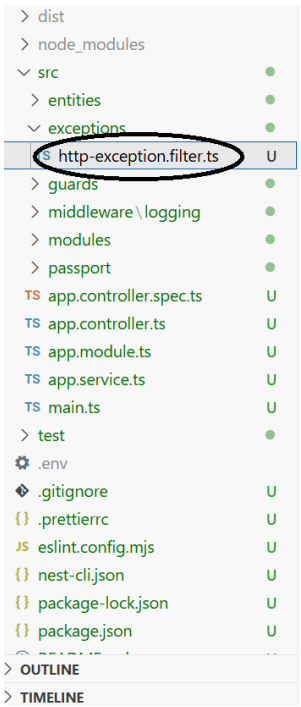
- Tiếp tục mở chương trình **Postman**, thiết lập phương thức **GET**, gõ đường dẫn: **localhost:3000/products/2** ***(Lưu ý:*** *sản phẩm có id=2 không có trong cơ sở dữ liệu).* Lúc này các bạn sẽ thấy thông báo **Exception** ra. Như vậy bài trước chúng ta đã sử dụng Exception sẵn có rồi.



12.1.3. Custom Exception

- Tiếp tục chúng ta cũng có thể tạo **Custom Exception** theo ý muốn của chúng ta, ví dụ thay đổi nội dung trả về lỗi bằng cách:

- Các bạn tạo một thư mục trong thư mục **src** và đặt tên là **exceptions**, và tạo trong thư mục **exception** một tập tin **http-exception.filter.ts** và viết code như sau:




```ts
// all-exceptions.filter.ts
import {ExceptionFilter,Catch,ArgumentsHost,HttpException,HttpStatus,} from '@nestjs/common';
import { Request, Response } from 'express';

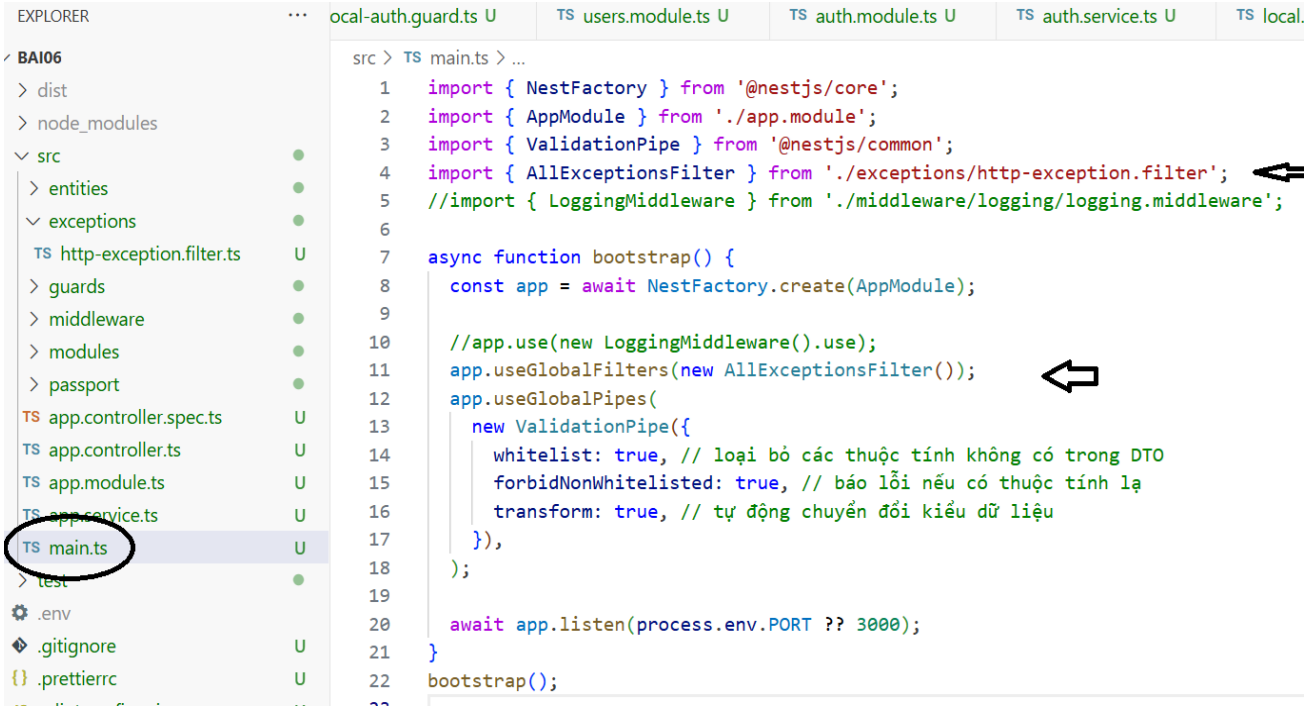
@Catch()
export class AllExceptionsFilter implements ExceptionFilter {
    catch(exception: any, host: ArgumentsHost) {
        const ctx = host.switchToHttp();
        const request = ctx.getRequest<Request>();
        const response = ctx.getResponse<Response>();

        const status =
            exception instanceof HttpException
                ? exception.getStatus()
                : HttpStatus.INTERNAL_SERVER_ERROR;

        const message =
            exception instanceof HttpException
                ? exception.getResponse()
                : exception.message || 'Lỗi hệ thống';

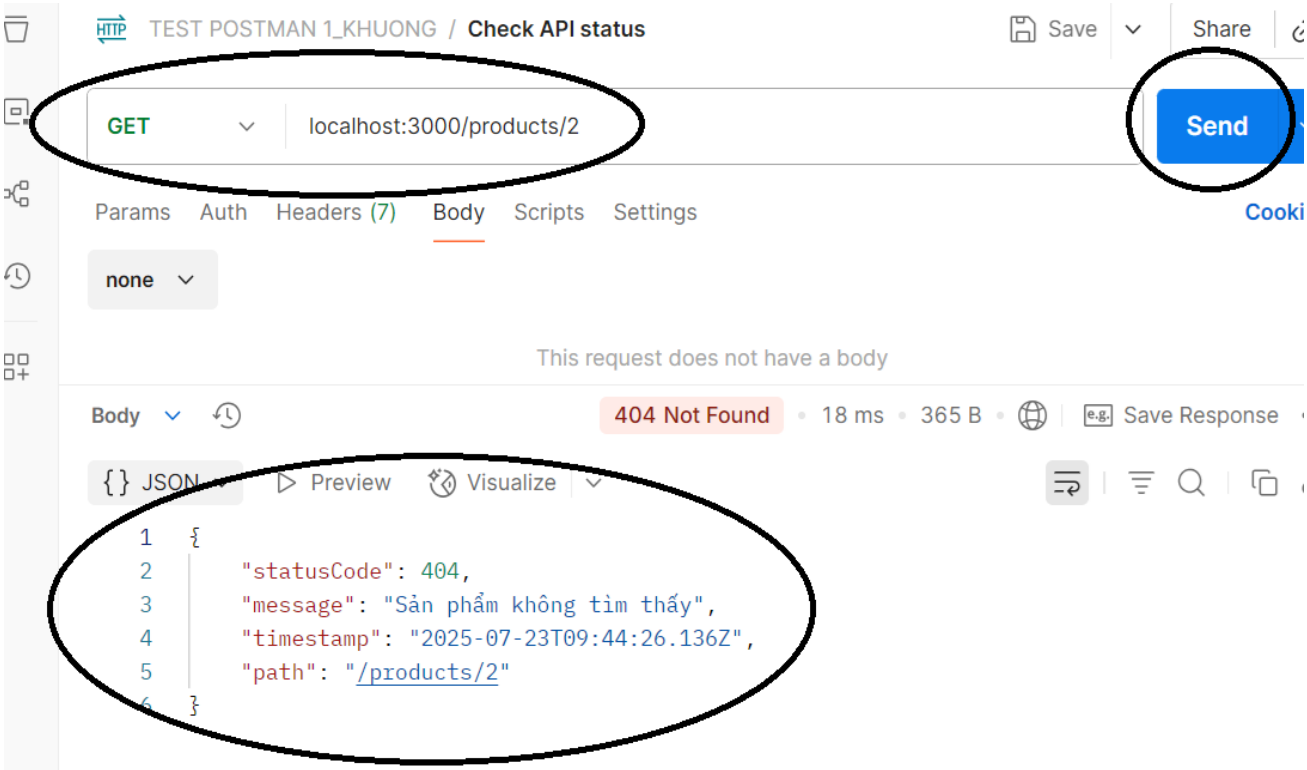
        response.status(status).json({
            statusCode: status,
            message: typeof message === 'string' ? message : (message as any).message,
            timestamp: new Date().toISOString(),
            path: request.url,
        });
    }
}
```

- Tiếp tục, để sử dụng **Exception** vừa tạo các bạn mở tập tin **main.ts** và viết thêm **code** như sau:



- Các bạn biên dịch chương trình: **npm run start:dev**

- Tiếp tục mở chương trình **Postman**, thiết lập phương thức **GET**, gõ đường dẫn: **localhost:3000/products/2** ***(Lưu ý:*** *sản phẩm có id=2 không có trong cơ sở dữ liệu)*. Lúc này các bạn sẽ thấy thông báo **Exception** ra theo yêu cầu đã tạo.



12.2. LOGGING TRONG NESTJS

12.2.1. Giới thiệu về Logging

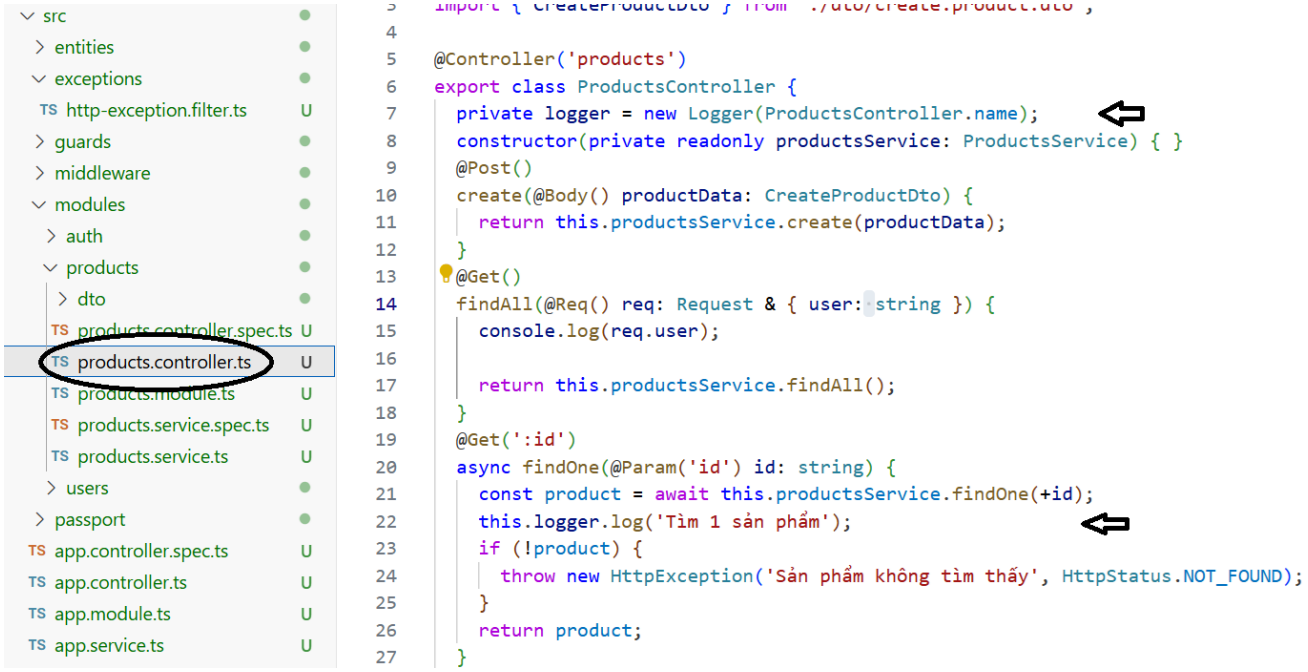
Logging là một phần rất quan trọng trong phát triển ứng dụng, giúp lập trình viên **theo dõi** hoạt động của hệ thống, phát hiện lỗi, ghi lại thông tin xử lý và phục vụ việc kiểm tra, bảo trì sau này.

NestJS cung cấp một **class Logger** tích hợp sẵn, có thể sử dụng ở bất cứ đâu trong ứng dụng:

- import { Logger } from '@nestjs/common';
- const logger = new Logger('MyService');
- logger.log('Log message');
- logger.error('Error message');
- logger.warn('Warning message');
- logger.debug('Debug message');
- logger.verbose('Verbose message');

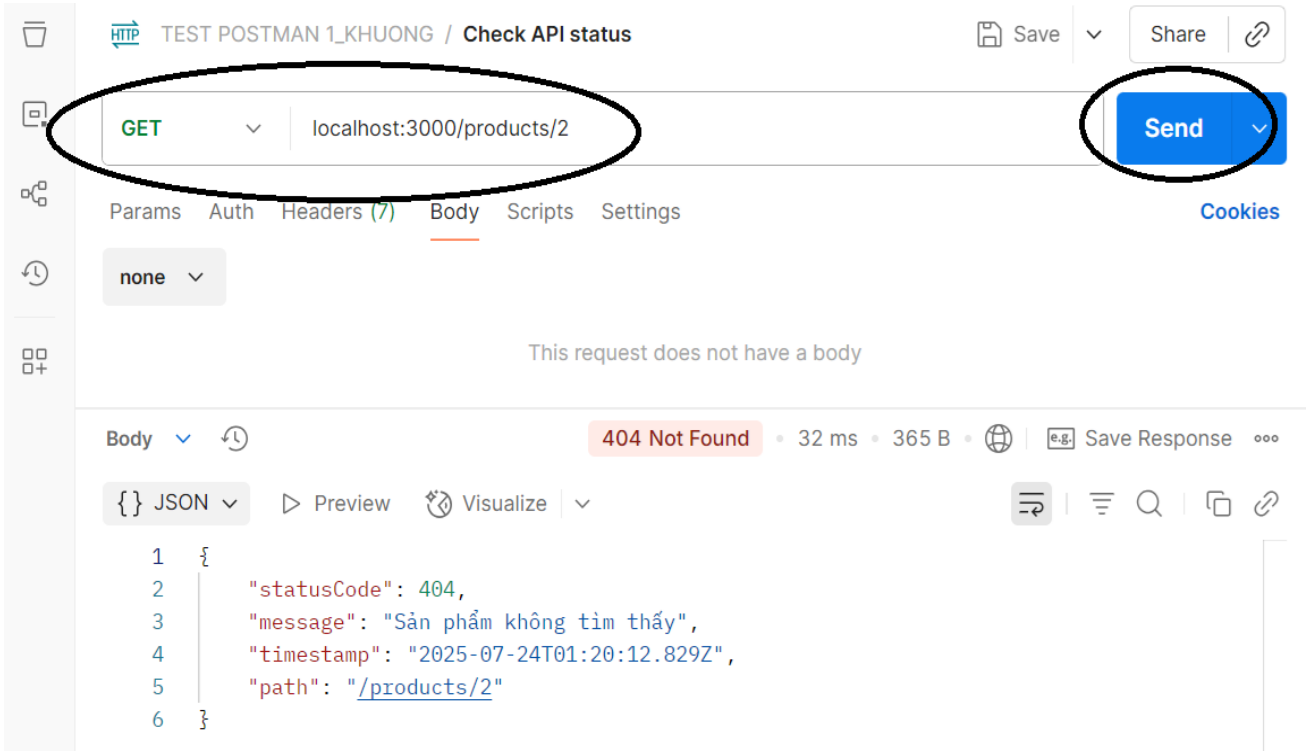
12.2.2. Ví dụ về Logging

- Các bạn mở lại dự án ở bài trước trong **VSCode**, sau đó mở tập tin **products.controller.ts** và viết thêm 2 dòng lệnh để đưa **Logging** vào như bên dưới:

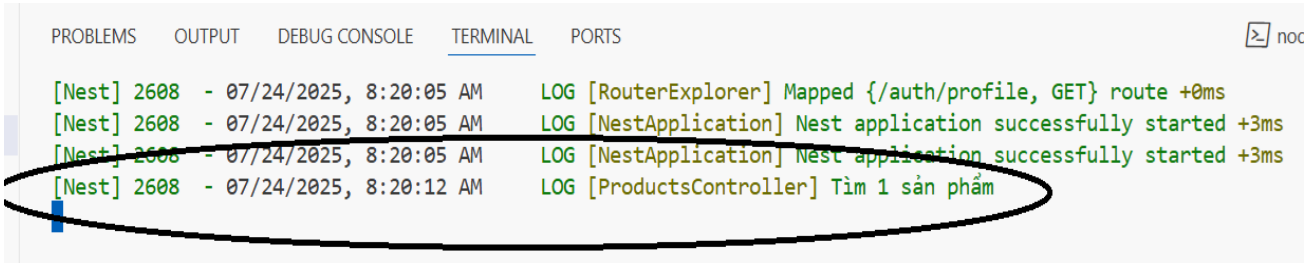


- Các bạn biên dịch chương trình: **npm run start:dev**

- Tiếp tục mở chương trình **Postman**, thiết lập phương thức **GET**, gõ đường dẫn: **localhost:3000/products/2**

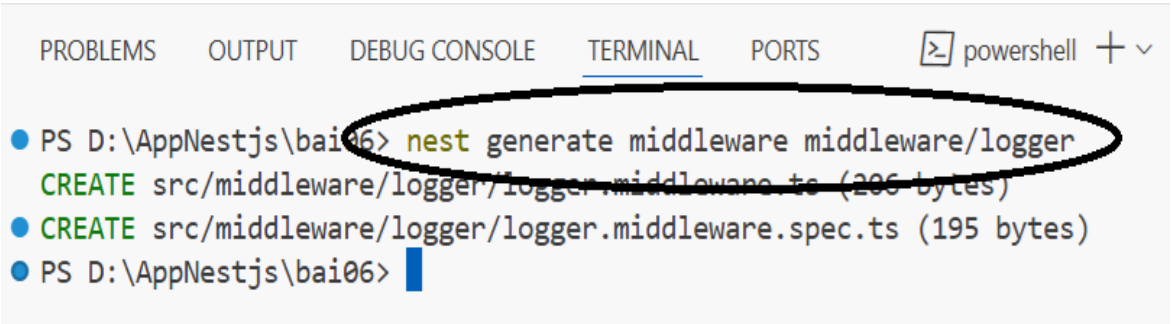


- Tiếp tục, các bạn mở **Terminal** của **VSCode** ra sẽ thấy **Logging** hiển thị "**Tìm 1 sản phẩm**", lưu đầy đủ thời gian, ngày giờ thực hiện thao tác Tìm sản phẩm.



12.2.3. Ghi Log vào file (Winston)

- Đầu tiên chúng ta muốn bất kì một **request** nào từ **Client** đều thấy được thì chúng ta tạo thêm 1 **middleware** bằng cách các bạn mở **Terminal** gõ dòng lệnh: **nest generate middleware middleware/logger**



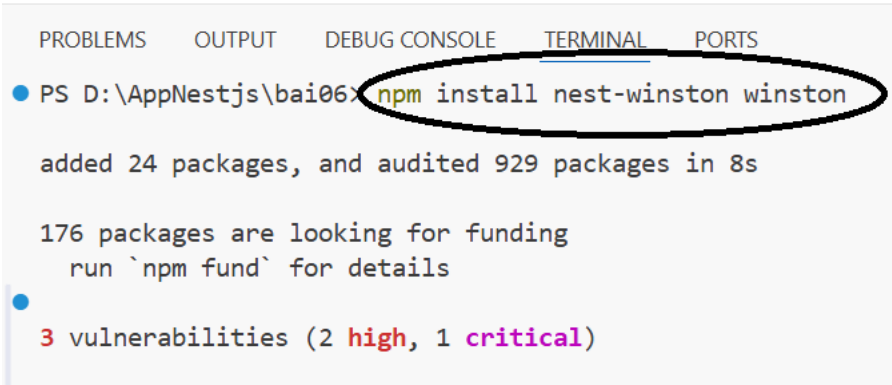
- Tiếp tục các bạn mở tập tin **logger.middleware.ts** và viết code như sau:



- Tiếp tục chúng ta sẽ gọi **middleware** này vào **main** để sử dụng bằng cách mở tập tin **main.ts** và gõ thêm dòng lệnh sau:



- Bây giờ chúng ta tiếp tục phần ghi dữ liệu **Logging** vào file bằng cách cài đặt thêm module **winston** vào dự án, các bạn mở **Terminal** gõ dòng lệnh: **npm install nest-winston winston**



- Tiếp tục, các bạn tạo trong thư mục **src** một thư mục mới đặt tên là: **logger**. Sau đó tạo trong thư mục **logger** một tập tin đặt tên là: **winston.logger.ts** và viết code như sau:

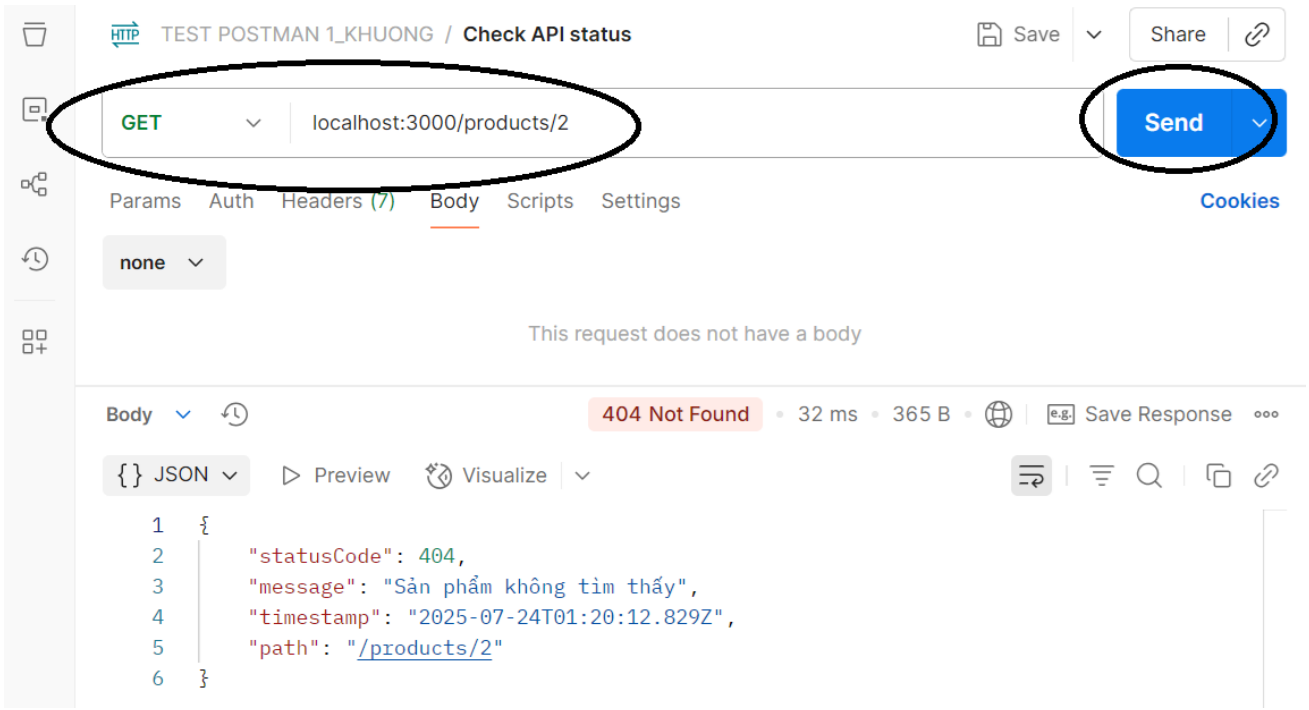


- Bây giờ sử dụng ghi **Logger** vào tập tin cho ứng dụng bằng cách mở tập tin **main.ts** và viết thêm **code** như sau:



- Các bạn biên dịch chương trình: **npm run start:dev**

- Tiếp tục mở chương trình **Postman**, thiết lập phương thức **GET**, gõ đường dẫn: **localhost:3000/products/2**



- Các bạn sẽ thấy xuất hiện một thư mục **logs** ở dự án, trong đó có 2 tập tin **combined.log** và **error.log** lưu các thông tin **request** yêu cầu từ **Client**

